

CCM310
Arquitetura de Software e Programação Orientada a Objetos

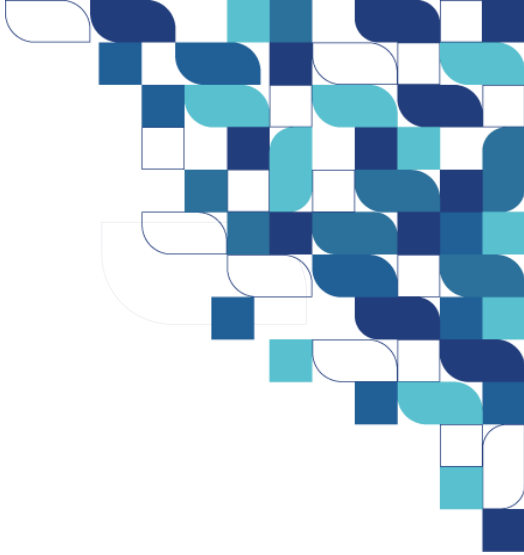
Profa. Dra. Gabriela Oliveira Biondi

Departamento de Ciência da Computação FEI

FEI

IHS JESUÍTAS BRASIL





Tratamento de Exceções

Aula 07

Exceções

- O momento ideal para detectar um erro é durante a compilação, antes mesmo de tentar executar o programa
- No entanto, nem todos os erros podem ser detectados em tempo de compilação
- O restante dos problemas deve ser tratado em tempo de execução

Exceções

- Uma exceção é um problema que acontece em tempo de execução!
- É um problema que, normalmente, não pode ser antecipado em um programa
- A exceção precisa ser tratada pelo programador

Exceções

- Exceções ocorrem quando um método detecta um problema e é incapaz de tratá-lo
- Exceções são geradas de três maneiras diferentes:
  - O sistema de suporte do tempo de execução pode gerar exceções em resposta a algum erro interno
  - Exceções padrões podem ser geradas por erros no código
  - Podemos gerar exceções manualmente

Exemplos de Exceção – Erros de Lógica

- Divisão por zero
(division by zero)

- Índice fora dos limites do vetor
(ArrayIndexOutOfBounds)

- Manipular objeto com valor nulo
(NullPointer)

Exemplos de Exceção – Motivos Externos

- Manipular/abrir arquivo não encontrado *(FileNotFound)*

- Tentar interagir com Banco de Dados não disponível *(SQLException)*

- Conectar a um servidor que não existe ou recusou a conexão *(ConnectException)*

Tratamento de Exceções

- No Java, o tratamento de exceções é gerenciado por cinco palavras-chave: **try**, **catch**, **throw**, **throws** e **finally**
  - **try:** é onde ficam as instruções com risco de falha
  - **catch:** é onde fica o código que trata a exceção
  - **throw:** lança uma exceção manualmente
  - **throws:** específica a exceção lançada para fora de um método
  - **finally:** é onde fica o código que deve sempre ser executado ao sair de um bloco try/catch

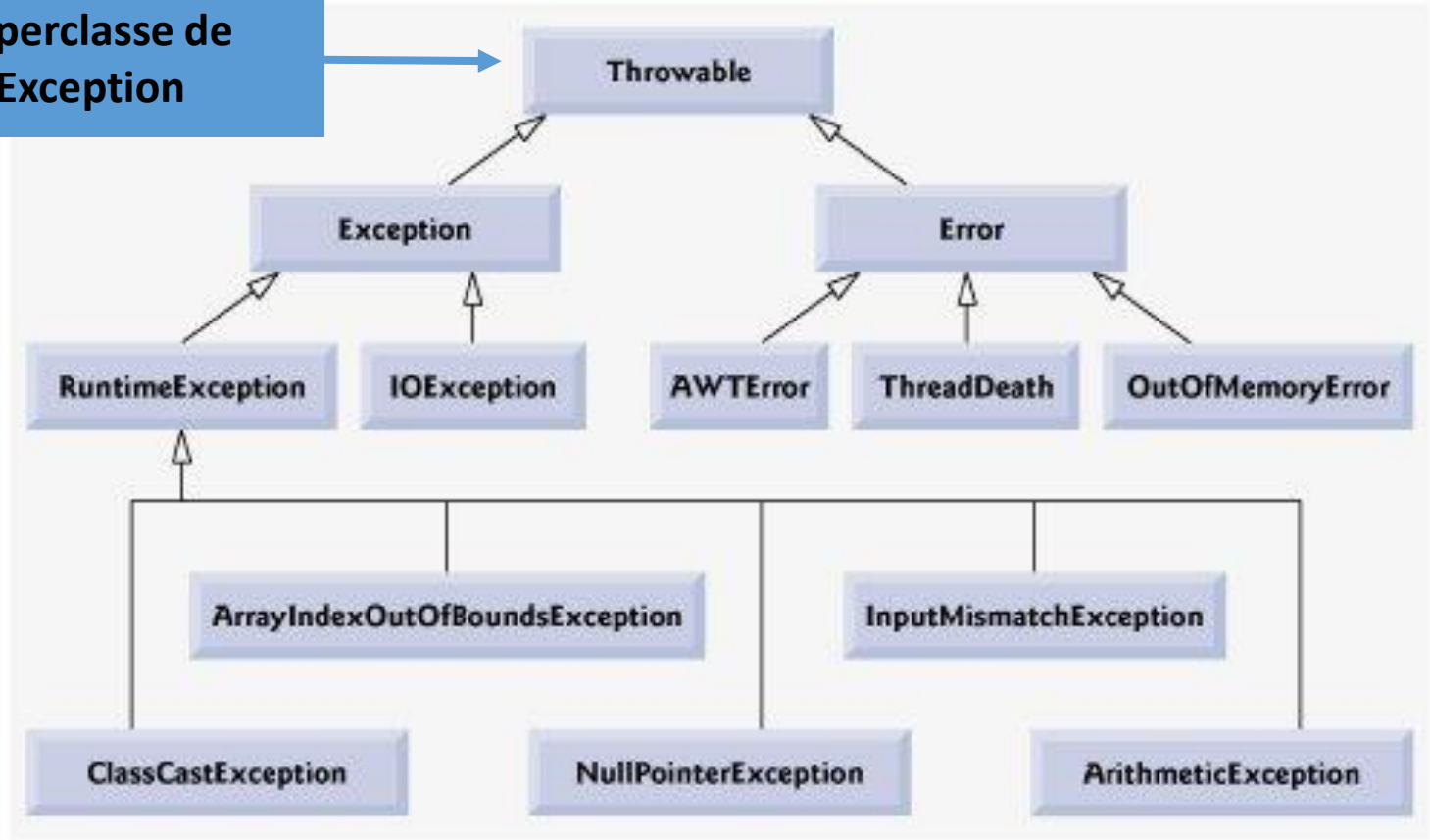
Classe Exception

- No Java, todas as classes de Exceção herdam, direta ou indiretamente, da classe **Exception**, formando uma hierarquia de herança

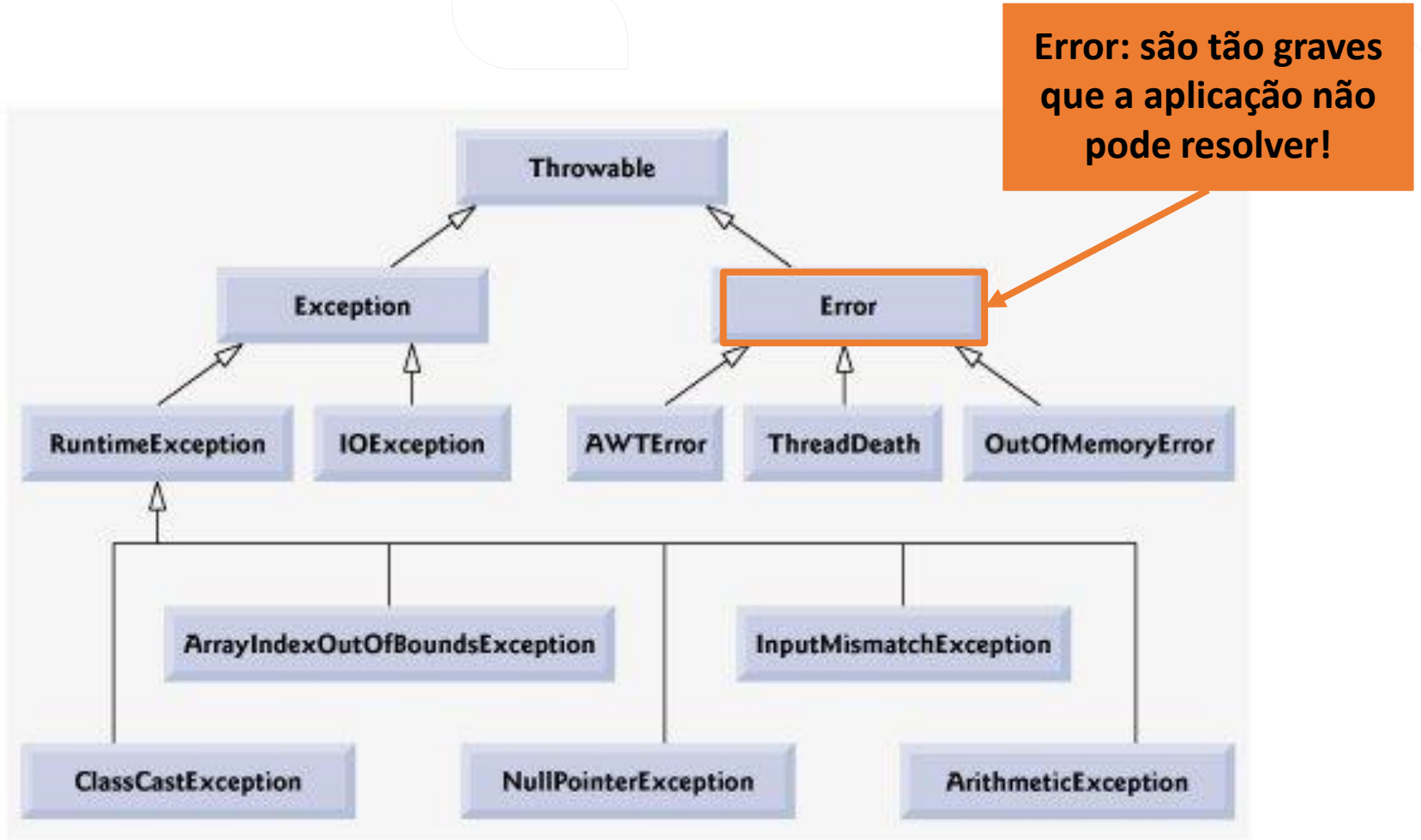
- Os programadores podem, além de utilizar as classes já prontas no Java, estender as classes existentes para criar suas próprias classes de Exceção

Classe Exception

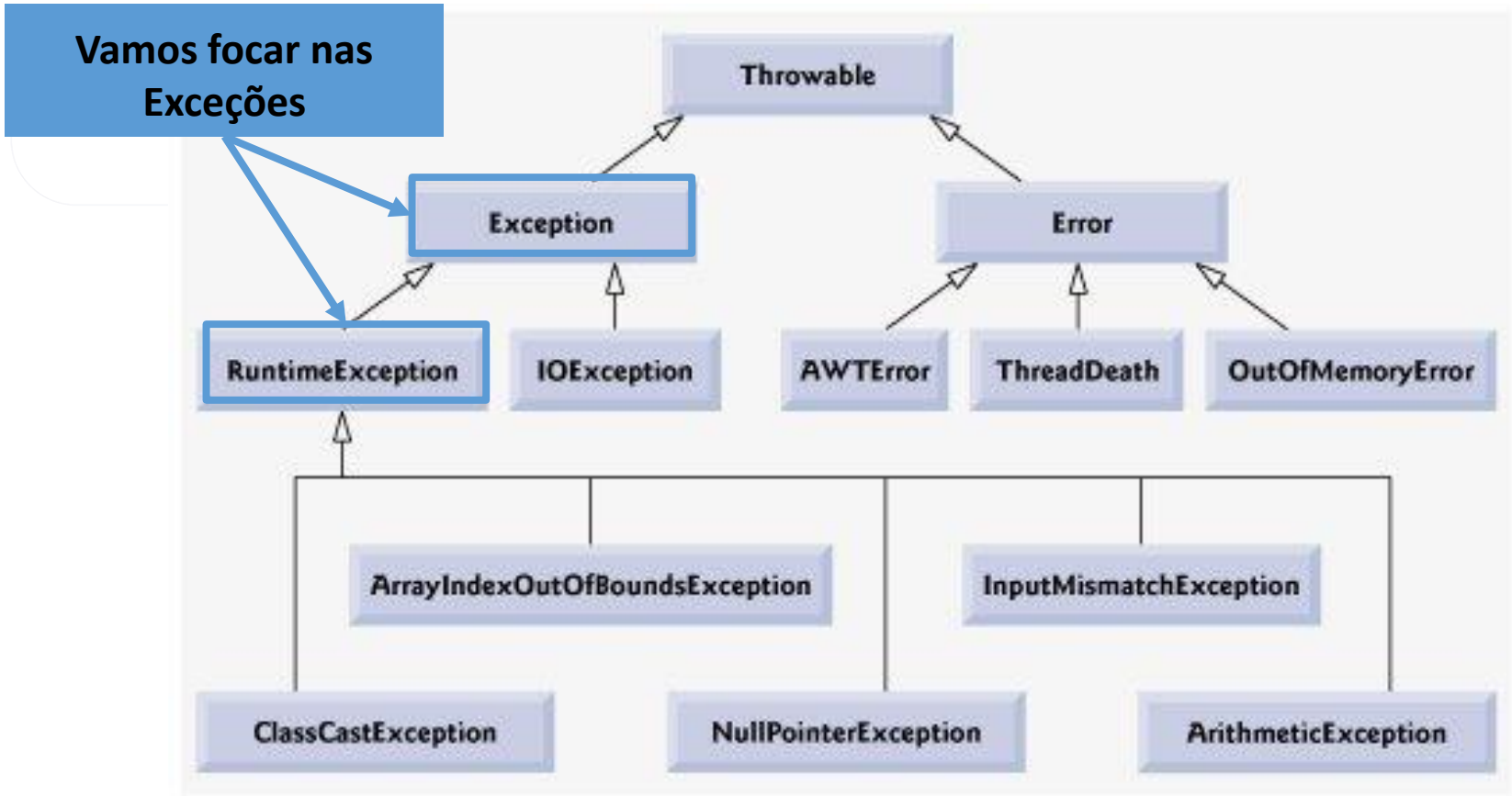
Superclasse de Exception



Classe Exception



Classe Exception



Exemplo de Exceção em Java

- **try:**
  - Código que contém um risco de falha
- **catch:**
  - Código para tratar uma exceção

Exemplo de Exceção em Java

```java
public class Main {
  public static void main(String args[]){
    Integer[] itens  = new Integer[10];

    for (int i = 0; i < itens.length; i++) {
     itens[i] = i;
    }

  for (int i = 0; i <= itens.length; i++) {
    try{
      System.out.println(itens[i]);
    }
    catch(ArrayIndexOutOfBoundsException e){
      System.out.println("deu exception no indice: "+ i);
    }
  }

  System.out.println("Fui executado!");
  }
}
```

try **e** ***catch*** **formam a base do tratamento de exceções!**

Elas funcionam em conjunto, ou seja, você não pode ter um ***catch*** **sem ter um** ***try***

Exemplo de Exceção em Java

- O que aconteceria se o array itens fosse *null*?
- Mesmo com o *try/catch* declarado, o tratamento da exceção não aconteceria, pois receberíamos um outro tipo de *exception*: ***java.lang.NullPointerException***
- Isso acontece porque estamos deixando claro, no parâmetro de nosso catch, que queremos capturar a exceção ***ArrayIndexOutOfBounds***

Exemplo de Exceção em Java

- Podemos capturar **Exception** no lugar de **ArrayIndexOutOfBoundsException**, isso faz com que qualquer exceção seja capturada:

```
1  public class Main {
2    public static void main(String args[]){
3      Integer[] itens  = null;
4
5      //for (int i = 0; i < itens.length; i++) {
6      // itens[i] = i;
7      //}
8
9    for (int i = 0; i <= 10; i++) {
10     try{
11         System.out.println(itens[i]);
12     }
13     catch(Exception e){
14       System.out.println("deu exception no indice: "+ i);
15     }
16   }
17
18   System.out.println("Fui executado!");
19   }
20 }
```

Exemplo de Exceção em Java

- Pode-se também utilizar **vários catch**. Para dar um tratamento diferenciado para cada tipo de problema:

```
1  for (int i = 0; i <= 10; i++) {
2    try{
3      System.out.println(itens[i]);
4    }
5    catch (ArrayIndexOutOfBoundsException e) {
6      System.out.println("deu exception no indice: "+ i);
7    }
8    catch (NullPointerException e) {
9      System.out.println("A array não foi instanciado!");
10   }
11 }
```

Exemplo de Exceção em Java

- **try:**
  - Código que contém um risco de falha
- **catch:**
  - Processa uma exceção:
  - O 1º catch trata uma exceção do tipo **InputMismatchException**
  - O 2º catch trata uma exceção do tipo **ArithmeticException**

```java
import java.util.InputMismatchException;
import java.util.Scanner;

class Main{

  public static int quociente(int num, int den) throws ArithmeticException
    return num / den;
  }

  public static void main(String args[]){
    Scanner scanner = new Scanner( System.in );
    int numerador = 1;
    int denominador = 1;
    do
    {
      try
      {
        System.out.println("Entre o numerador: ");
        numerador = scanner.nextInt();
        System.out.println("Entre o denominador: ");
        denominador = scanner.nextInt();

        int result = quociente(numerador, denominador);
        System.out.println("Resp: " + result);
      }
      catch (InputMismatchException inpmisexp)
      {
        System.err.printf("Exception: %s\n", inpmisexp);
        scanner.nextLine();
        System.out.println("Você deve entrar com os numeros novamente: ");
      }
      catch (ArithmeticException arithm)
      {
        System.err.printf("Exception: %s\n", arithm);
        System.out.println("Zero não é um denominador valido ");
      }
    } while (numerador != 0);
  }
}
```

Exemplo de Exceção em Java

- Cláusula **throws**:
  - Especifica as exceções que o método lança

```
1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  class Main{
5
6    public static int quociente(int num, int den) throws ArithmeticException {
7      return num / den;
8    }
9
```

Exemplo de Exceção em Java

- Cláusula **finally**:
  - Existem momentos em que precisamos executar alguma ação após um **try** ou **catch**, independente se houve ou não uma **exception**
  - Nesses casos, podemos utilizar uma terceira cláusula chamada **finally**

Exemplo de Exceção em Java

- Cláusula **finally**:

```
try {
    // código de risco
} catch (NullPointerException e) {
    // tratando NPE
} catch (Exception e) {
    // tratando Exception
} finally {
    // alguma ação importante
}
```

Exemplo de Exceção em Java

- Cláusula **finally**:
  - As instruções do bloco **finally** serão executadas sempre, independente de tudo ter ido bem ou de alguma exceção ter acontecido
  - Esse recurso é muito comum quando estamos trabalhando com conexão do banco de dados ou leitura/escrita de arquivos. Independentemente de qualquer coisa, depois de usar uma conexão, por exemplo, sempre queremos fechá-la

Exemplo de Exceção em Java

- Cláusula **finally**:

```
12  public class Aula7ex2 {
13
14      public static int excecao(){
15          try{
16              int n = 7/0;
17              System.out.println( x: "try");
18          }
19          catch(Exception e){
20              System.out.println( x: "catch");
21              //return 0;
22          }
23          finally{
24              System.out.println( x: "finally");
25          }
26          System.out.println( x: "fora try-catch-finally");
27          return 0;
28      }
29
30      public static void main(String[] args) {
31          excecao();
32          System.out.println( x: "main");
33      }
34  }
```

Exemplo de Exceção em Java

- Criando **Exceptions** com **throw**
- **throw:** lança uma exceção manualmente

```java
class IntException extends RuntimeException{
  public IntException(String message){
    super(message);
  }
}
```

```java
class Main{

  public static void Xtest(int test){
      System.out.println("Inside Xtest: " + test);
      if (test == 1)
        throw new IntException("Erro!");
  }

  public static void main(String args[]){
    System.out.println("Start\n");
    try
    {
      System.out.println("Inside block try");
      Xtest(0);
      Xtest(1);
      Xtest(2);
    }
    catch(RuntimeException e)
    {
      System.out.println("Caught an exception " + e);
    }

    System.out.println("Continua codigo");
    System.out.println("END");
  }
}
```

Exemplo de Exceção em Java

- Criando **Exceptions** com **throw**
- Vamos supor que precisamos da **classe Livro**
- A classe deve receber um objeto Autor pelo construtor (agregação)
- O objeto autor não pode ser nulo

Exemplo de Exceção em Java

- Criando **Exceptions** com **throw**

```java
public class AutorNuloException extends RuntimeException{
  public AutorNuloException(String mensagem) {
    super(mensagem);
  }
}
```

```java
class Main{
  public static void main(String args[]){
    try
    {
      Livro livro = new Livro(null);
    }
    catch(RuntimeException e)
    {
      e.printStackTrace();
    }

    System.out.println("continua código");
  }
}
```

```java
class Livro{
  public Livro(Autor autor) {
    if (autor == null) {
      throw new AutorNuloException("O Autor do Livro não pode ser nulo");
    }
    this.autor = autor;
    isbn = "000-00-00000-00-0";
  }
  private Autor autor;
  private String isbn;
}
```

Por que herdar das classes da biblioteca padrão?

- Exemplo:
- No seu código você pode ter diversos tipos de falha
- Se todas as suas classes herdarem de **Exception** (ou de suas classes derivadas) você só precisa ter um **catch** para todas as exceções lançadas
- Do contrário você vai precisar de um **catch** para cada uma

Tratamento de exceção *vs.* if/else

- Exceções não devem ser usadas para controle de fluxo
- Exceções devem ser usadas realmente em exceções!
- Naquele lugar na qual nada deveria / poderia dar errado, mas, se der, você pode ter um grande problema!
- Problemas simples e de escopo local podem ser tratados com *if / else*

Exercício 01

- Crie um comportamento de retomada usando um *while* que repete até que uma exceção não seja mais lançada.

Exercício 02

- Declare um objeto e inicialize-o como *null*. Tente chamar um método por meio deste objeto. Agora envolva o código em uma cláusula *try-catch* para capturar a exceção.

Exercício 03

- Crie uma classe com o método *main()* que lança um objeto da classe *Exception* dentro de um bloco *try*. Dê ao construtor de *Exception* um argumento *String*. Capture a exceção dentro de uma cláusula *catch* e imprima o argumento *String*. Adicione a cláusula *finally* com uma mensagem.

Exercício 04

- Crie sua própria classe de exceção usando Herança. Escreva um construtor para esta classe que recebe um argumento *String* e o armazena dentro do objeto. Escreva um método que exibe a *String* armazenada. Crie uma cláusula *try-catch* para exercitar sua nova exceção.

Exercício 05

- Crie três novos tipos de exceções. Escreva uma classe com um método que lança todos os três. No *main()*, chame o método, mas use apenas uma única cláusula *catch* que deve capturar todos os três tipos de exceções.

Exercício 06

- Faça uma *classe Pessoa*, com um construtor de 4 parâmetros: nome, sobrenome, idade e CPF. O programa deve gerar uma exceção específica (crie uma classe de exceção) caso o CPF seja digitado com caracteres especiais (ponto e hífen). A mensagem exibida, neste caso, deve indicar qual foi o erro.

- No programa principal o objeto pessoa será instanciado com os 4 dados sendo digitados pelo usuário. No caso de erro, o CPF deve ser digitado novamente.

Exercício 07

- Faça uma classe *Login* com os atributos usuario e senha. A classe deve ter também os métodos *get* e *set*, construtor parametrizado e o método *logar(user, password)*, que recebe um usuário e uma senha e compara com os atributos da classe. Caso as informações não sejam corretas, uma exceção deve ser lançada.

- No programa principal, instancie um objeto da classe *Login* e utilize o método *logar(user, password)* para verificar usuario e senha.

Exercício 08

- Faça uma *classe Carro* com os atributos velocidade e velocidadeMaxima. A classe deve ter também os métodos *get* e *set*, construtor padrão e parametrizado e os métodos acelerar(valor) e frear(valor), onde valor é o incremento ou o decremento na velocidade. Caso a velocidade fique menor do que zero ou maior do que a velocidadeMaxima, exceções devem ser lançadas.

- No programa principal, instancie um objeto da *classe Carro* e teste a aplicação.

Obrigada pela sua participação, nos vemos na próxima aula!

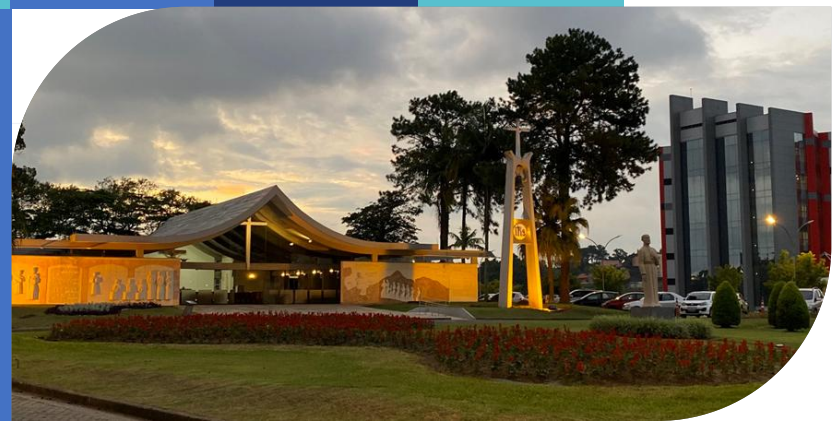
:)

JESUÍTAS BRASIL









FEI edu.br

JESUÍTAS BRASIL